Every Big Model Starts with a Small Ablation

The Smol Training Playbook

experiment -> fail -> learn

On HumanEval and MBPP, we conduct zero-shot and few-shot evaluations, respectively.

Model	Size	ArXiv Corpus	English Benchmarks					Chinese Benchmarks		
			GSM8K	MATH	OCW	SAT	MMLU STEM	CMATH	Gaokao MathCloze	Gaokao MathQA
DeepSeek-LLM	1.3B	No Math Training	2.9%	3.0%	2.9%	15.6%	19.5%	12.3%	0.8%	17.9%
		MathPile	2.7%	3.3%	2.2%	12.5%	15.7%	1.2%	0.0%	2.8%
		ArXiv-RedPajama	3.3%	3.4%	4.0%	9.4%	9.0%	7.4%	0.8%	2.3%
DeepSeek-Coder-Base-v1.5	7B	No Math Training	29.0%	12.5%	6.6%	40.6%	38.1%	45.9%	5.9%	21.1%
		MathPile	23.6%	11.5%	7.0%	46.9%	35.8%	37.9%	4.2%	25.6%
		ArXiv-RedPajama	28.1%	11.1%	7.7%	50.0%	35.2%	42.6%	7.6%	24.8%

Table 8 | Effect of math training on different arXiv datasets. Model performance is evaluated with few-shot chain-of-thought prompting.

https://arxiv.org/abs/2402.03300

baselines

Llama 2 -> Llama3

DeepSeek-V3 -> Kimi K2

- Match your constraints, aligning with your deployment target and use case
- Be proven at scale (demonstrated in multi-trillion-token runs at similar or larger sizes)
- Be well-documented, with known hyperparameters that have been proven to work in open models
- Have good framework support (ideally, it should be supported in the training frameworks you are considering and the inference frameworks you are planning to use)

solid standard baseline >> **flaky** artisan cool bleeding edge rocketship

The discipline that keeps you on track is *derisking* : Never change anything unless you've tested that it helps.

> 📍 **What counts as derisked?**
>
> A change is derisked when testing shows it either improves performance on your target capabilities or provides a meaningful benefit (e.g., faster inference, lower memory, better stability) without hurting performance beyond your acceptable limits.

🎯 **Strategic experimentation**

Knowing how to run experiments isn't enough; you need to know which experiments are worth running. Ask yourself two questions before testing any modification:

- Will this help my specific use case?
- Will this optimize my training?

If a modification doesn't clearly address either goal, skip it.

Framework	Features	Battle-Tested	Optimized
Megatron-LM	✅ Extensive	✅ Kimi K2, Nemotron	✅ Pioneers of 3D parallelism
DeepSpeed	✅ Extensive	✅ BLOOM, GLM	✅ Pioneers of ZeRO & 3D parallelism
TorchTitan	⚡ Growing feature set	⚠️ Newer but tested by PyTorch team	⚡ Optimized for dense models, MoE impr
Nanotron	🎯 Minimal, tailored for HF pretraining	✅ Yes (StarCoder, SmolLM)	✅ Optimized (UltraScale Playbook)

Framework	Features	Battle-Tested	Optimized
Megatron-LM	✅ Extensive	✅ Kimi K2, Nemotron	✅ Pioneers of 3D parallelism
DeepSpeed	✅ Extensive	✅ BLOOM, GLM	✅ Pioneers of ZeRO & 3D parallelism
TorchTitan	⚡ Growing feature set	⚠️ Newer but tested by PyTorch team	⚡ Optimized for dense models, MoE impr
Nanotron	🎯 Minimal, tailored for HF pretraining	✅ Yes (StarCoder, SmolLM)	✅ Optimized (UltraScale Playbook)

- https://github.com/mosaicml/llm-foundry - Databricks
- https://github.com/marin-community/levanter - JAX + TPUs
- https://github.com/facebookresearch/lingua - Meta Research
- https://github.com/allenai/OLMo-core/ - AI2

Ultimately, your choice will depend on your team's expertise, your target features, and how much time you're willing to invest in development versus using the most production-ready option.

Ultimately, your choice will depend on your team's expertise, your target features, and how much time you're willing to invest in development versus using the most production-ready option.

The goal of ablations is to run experiments at a small scale and get results we can confidently extrapolate to our final production run.

The goal of ablations is to run experiments at a small scale and get results we can confidently extrapolate to our final production run.

Also, experiment in isolation?

- Train fewer tokens
- Train smaller proxy (scaling laws? transferability?)

- Train fewer tokens
- Train smaller proxy (scaling laws? transferability?)

One key question is whether these small-scale findings actually transfer. In our experience, if something hurts performance at small scale, you can confidently rule it out for large scale. But if something works at small scale, you'll want to make sure you've trained on a reasonable number of tokens to conclude with high probability that these findings will extrapolate to larger scales. The longer you train and the closer the ablation models are to the final model, the better.

- Train fewer tokens
- Train smaller proxy (scaling laws? transferability?)

One key question is whether these small-scale findings actually transfer. In our experience, if something hurts performance at small scale, you can confidently rule it out for large scale. But if something works at small scale, you'll want to make sure you've trained on a reasonable number of tokens to conclude with high probability that these findings will extrapolate to larger scales. The longer you train and the closer the ablation models are to the final model, the better.

☝️ **Modify one thing at a time**

Change only one variable per ablation, while keeping everything else constant. If you change multiple things and performance improves, you won't know what caused it. Test modifications individually, then combine successful ones and reassess.

WARNING!
Some architectural changes can significantly alter the parameter count
Adjust hyperparameters accordingly!

Vocabulary Size 128 k

Hidden Size 2048

Layers 16

Attention Heads 32

KV Heads 32

Intermediate Size 8192

Tie Embeddings

Yes

1.34 B

Parameters

EMBEDDINGS

Input + Output Projection

262 M

128 k × 2048

vocab_size × hidden_size

ATTENTION LAYERS

Q, K, V, O projections

268 M

16 × 2048² × 4

layers × hidden_size² × 4

FEED FORWARD

Up, Gate, Down projections

805 M

16 × 2048 × 8192 × 3

layers × hidden_size × intermediate_size × 3

LAYER NORMS

Input + Attention norms

68 K

16 × 2048 × 2

layers × hidden_size × 2

ATTENTION TYPE	EMBEDDING STRATEGY	PARAMS PER LAYER
MHA	Tied	67 M

evals >> loss

- **Monotonicity:** The benchmark scores should consistently improve as models train longer.
- **Low noise:** When we train models with the same setup but different random seeds, the benchmark scores shouldn't vary wildly.
- **Above-random performance:** Many capabilities only emerge later in training, so tasks that show random-level performance for extended periods aren't useful for ablations. This is the case, for example, for MMLU in multiple choice format, as we will explain later.
- **Ranking consistency:** If one approach outperforms another at early stages, this ordering should remain stable as training continues.

- Multiple choice format (MCF)
- Cloze formulation (CF)
- Free-form generation (FG)

- Multiple choice format (MCF)
- Cloze formulation (CF)
- Free-form generation (FG)

Research has also shown that models struggle with MCF early in training, making CF better for early signal (Du et al., 2025; Gu et al., 2025; J. Li et al., 2025). We thus use CF for small ablations and integrate MCF in the main run (as it gives better signal at later stages of training). To score a model's answer in sequence likelihood evaluations like CF, we compute

For *data ablations* , we take the opposite approach: We fix the architecture and systematically vary the data mixtures to understand how different data sources affect model performance.

Phase	GPUs	Days	GPU Hours
Main pretraining run	384	30	276,480
Ablations (pretraining)	192	15	69,120
Ablations (mid-training)	192	10	46,080
Training reset / debugging	384 / 192	3 / 4	46,080
Total cost	-	-	**437,760**

Validate your evaluation suite. Before training any models, make sure your evaluation suite can reproduce the published results of models you will compare against. If any benchmarks are generative in nature (e.g., GSM8k), be extra paranoid and manually inspect a few samples to ensure that the prompt is formatted correctly and that any post-processing is extracting the correct information. Since evals will guide every single decision, getting this step right is crucial for the success of the project!

Test every change, no matter how small. Don't underestimate the impact of that seemingly innocent library upgrade or the commit that "only changed two lines." These small changes can introduce subtle bugs or performance shifts that will contaminate your results. You need a library with a strong test suite on the cases that matter to you to avoid regression.

Change one thing at a time. Keep everything else identical between experiments. Some changes can interact with each other in unexpected ways, so you want to assess the individual contribution of each change first, then try combining them to see their overall impact.

Train on enough tokens and use sufficient evaluations. As we mentioned earlier, you need to make sure you have good coverage in your evaluation suite and train long enough to get reliable signal. Cutting corners here will lead to noisy results and bad decisions.